

Aula 1 — Ecossistema .NET e Setup Profissional

Carga horária: 4 horas

Objetivo da aula:

Compreender o ecossistema moderno do .NET, configurar corretamente o ambiente de desenvolvimento e criar o primeiro projeto profissional ASP.NET Core seguindo boas práticas iniciais.

PARTE 1 — TEORIA (≈ 2 horas)

1 Evolução do .NET

.NET Framework → .NET Core → .NET 8

◆ .NET Framework (até 4.8)

- Exclusivo para **Windows**
- Fortemente acoplado ao IIS
- Pouca flexibilidade para cloud e containers
- Legado (manutenção apenas)

◆ .NET Core (2016–2020)

- **Cross-platform** (Windows, Linux, macOS)
- Open Source
- Alto desempenho
- Base para cloud, microservices e Docker

◆ .NET 5+ → .NET 6 → .NET 7 → .NET 8 (LTS)

- Unificação total do ecossistema

- Performance otimizada
- Minimal APIs
- Melhor suporte a SPA (React, Angular)
- .NET 8 é LTS (Long Term Support)

👉 Conclusão prática:

Todo novo projeto profissional deve usar **.NET 8 LTS**.

2 CLI, SDK e LTS

◆ SDK (.NET Software Development Kit)

Inclui:

- Compilador C#
- CLI (dotnet)
- Templates de projetos
- Runtime

◆ CLI (dotnet)

Permite:

- Criar projetos
- Compilar
- Executar
- Publicar
- Criar migrations

Exemplo:

```
dotnet new webapp dotnet run dotnet publish
```

◆ LTS (Long Term Support)

- Suporte por 3 anos
- Ideal para produção
- Ex: .NET 6 LTS, .NET 8 LTS

3 Visual Studio vs VS Code

Visual Studio	VS Code
IDE completa	Editor leve
Melhor para iniciantes	Melhor para times fullstack
Debug avançado	Extensível
Windows	Multiplataforma

👉 Recomendação didática:

- **Visual Studio 2022** para backend .NET
- **VS Code** para React/Blazor WASM

4 Estrutura de Projetos ASP.NET Core

Estrutura padrão MVC:

```
MeuProjeto.Web |— Controllers |— Models |— Views |— wwwroot |— appsettings.json |— Program.cs
```

Estrutura profissional (camadas):

MeuProjeto |— MeuProjeto.Web → UI / Controllers |— MeuProjeto.Application → Regras de negócio |— MeuProjeto.Domain → Entidades
|— MeuProjeto.Infra → Banco de dados

5 Padrão MVC vs SPA

◆ MVC (Model-View-Controller)

- Renderização no servidor
- Razor Views
- Ideal para sistemas administrativos

◆ SPA (React / Blazor WASM)

- Renderização no cliente
- Comunicação via API
- Experiência moderna

👉 Nesta disciplina:

- **Início com MVC/Razor**
 - Evolução para **Blazor e React**
-

🧪 PARTE 2 — PRÁTICA (≈ 2 horas)

1 Instalação do SDK .NET

✓ Passo 1 — Download

Site oficial:

<https://dotnet.microsoft.com/download>

Instalar:

- **.NET 8 SDK**
- **Visual Studio 2022**
 - ASP.NET e desenvolvimento Web
 - Desenvolvimento .NET Desktop

✓ Verificação

```
dotnet --version
```

Saída esperada:

```
8.0.xxx
```

2 Criação do Projeto ASP.NET Core Web App

✓ Criar solução

```
dotnet new sln -n Aula01DotNet cd Aula01DotNet
```

✓ Criar projeto Web (MVC)

```
dotnet new mvc -n Aula01.Web dotnet sln add Aula01.Web
```

3 Estrutura Inicial do Projeto

Aula01.Web |— Controllers | └─ HomeController.cs |— Models |— Views | └─ Home | └─ Index.cshtml |— wwwroot |— Program.cs

4 Análise do Program.cs (comentado)

```
var builder = WebApplication.CreateBuilder(args); // Adiciona serviços ao container de injeção de dependência
builder.Services.AddControllersWithViews(); var app = builder.Build(); // Configuração do pipeline HTTP
if (!app.Environment.IsDevelopment()) { // Página de erro padrão em produção app.UseExceptionHandler("/Home/Error"); } // Serve arquivos estáticos (CSS, JS, imagens) app.UseStaticFiles(); app.UseRouting(); // Autorização (usado futuramente)
app.UseAuthorization(); // Configuração de rotas MVC app.MapControllerRoute( name: "default", pattern: "{controller=Home}/{action=Index}/{id?}"); app.Run();
```

5 Criando um Controller (comentado)

📁 Controllers/HomeController.cs

```
using Microsoft.AspNetCore.Mvc; namespace Aula01.Web.Controllers { public class HomeController : Controller { // Action padrão
public IActionResult Index() { // Envia dados para a View ViewBag.Mensagem = "Primeira aplicação ASP.NET Core"; return View(); }
} }
```

6 Criando a View Razor

📁 Views/Home/Index.cshtml

```
@{ ViewData["Title"] = "Home"; } <h1>@ViewBag.Mensagem</h1> <p> Aplicação ASP.NET Core rodando com sucesso! </p>
```

7 Organização Inicial em Camadas (simulada)

Criar pastas para boas práticas:

```
Aula01.Web |— Controllers |— Models |— Services ← regras futuras |— Views
```

Exemplo de Service:

```
namespace Aula01.Web.Services { public class MensagemService { public string ObterMensagem() { return "Mensagem vinda da camada de serviço"; } } }
```

Registrar no Program.cs :

```
builder.Services.AddScoped<MensagemService>();
```

8 Primeiro Deploy Local

✓ Executar aplicação

```
dotnet run
```

Acessar no navegador:

```
https://localhost:5001
```

Ou:

`http://localhost:5000`

Resultado Esperado

- Ambiente configurado
 - Projeto ASP.NET Core rodando
 - Entendimento do pipeline
 - Base sólida para Razor, Blazor e React
-

Atividade Avaliativa da Aula

Desafio prático:

1. Criar novo Controller `SobreController`
2. Criar View `Sobre`
3. Exibir informações do projeto
4. Organizar código seguindo padrão visto